

Workspaces and Tools

Section: 2 of 11 | Subtopic: 3 of 3 — Workspaces & Tools | Duration: Part of 4 – 5 hours | [Theory](#)

[Practical](#)

Topics covered: What is a monorepo · npm Workspaces · Yarn overview · pnpm overview · Choosing the right tool

Learning Objectives — This Subtopic

- Understand what a monorepo is and the problem it solves.
- Set up and use npm Workspaces to manage multiple packages in a single repository.
- Understand how packages reference each other inside a workspace.
- Understand at a high level what Yarn and pnpm are and why they exist as alternatives to npm.
- Know when to use npm, Yarn, or pnpm, and recognise their commands when you encounter them in existing projects.

1. The Problem Workspaces Solve

To understand workspaces, you first need to understand the situation that makes them necessary. Consider a realistic scenario: you are building a full-stack web application that has three separate pieces of code:

- A **frontend** — the browser-facing React application
- A **backend** — the Node.js/Express API server
- A **shared library** — utility functions and TypeScript types used by *both* the frontend and backend (e.g., input validation rules, shared data models)

Each of these is technically a separate Node.js package with its own `package.json`. The question is: *how do you organise them?*

Option A — Separate Repositories (Polyrepo)

Store each package in its own separate Git repository. This is simple at first, but it creates real problems as the project grows:

PROBLEMS WITH SEPARATE REPOS

To use the shared library in the frontend, you must publish it to npm, then install it as a dependency. Every change to the shared code requires: edit → publish → update version → reinstall in both frontend and backend.

Making a single change that touches all three parts requires three separate pull requests in three repositories — hard to review and easy to get out of sync.

Running tests across all packages requires switching between three repositories and running commands in each separately.

WHAT WE WANT INSTEAD

All packages live in one Git repository. Changes to shared code are immediately visible to other packages — no publish cycle required.

A single pull request can change the shared library, the frontend that uses it, and the backend that uses it — all reviewed and merged together.

Run all tests across all packages with a single command from the root.

Keeping all related packages together in a single repository is called a **monorepo** (short for monolithic repository). Large companies like Google, Meta, and Microsoft manage enormous monorepos. In the Node.js world, many popular projects — React, Babel, Jest, Vue — are monorepos.

Analogy

Think of a clothing company that makes shirts, trousers, and jackets. The *polyrepo* approach is like having three separate factories in three different cities — each makes one product, and they ship raw materials between cities every time something changes. The *monorepo* approach is one large factory where all products are made under one roof — the fabric department, cutting room, and assembly line all share the same space and can hand materials to each other instantly.

2. npm Workspaces

npm Workspaces is a built-in feature of npm (available since npm v7, which comes with Node.js 15+) that provides official monorepo support. It allows one `package.json` at the root of your repository to declare and manage multiple sub-packages (called *workspaces*) simultaneously.

The key behaviours workspaces enable:

- A single `npm install` at the root installs dependencies for *all* packages.
- All packages share a single `node_modules` at the root (hoisting), avoiding duplicate installations.
- Packages can reference each other using their package names — npm automatically creates symlinks between them.
- You can run scripts in all packages or specific packages from the root with one command.

Stage 1 — The Structure Before Workspaces

First, understand what the folder structure looks like when you have multiple related packages *without* workspaces. Each package is completely independent and has its own `node_modules`:

```
my-app/                                ← just a folder, not a project
├── packages/
│   ├── frontend/
│   │   ├── package.json               ← { "name": "frontend" }
│   │   └── node_modules/              ← dependencies installed here
│   ├── backend/
│   │   ├── package.json               ← { "name": "backend" }
│   │   └── node_modules/              ← same dependencies duplicated here
│   └── shared/
│       ├── package.json               ← { "name": "@my-app/shared" }
│       └── node_modules/              ← more duplicates

```

(no root package.json)

With this structure, if `frontend` and `backend` both use `express@4.18.2`, it is downloaded and stored twice — one copy in each `node_modules`. To run the whole project you must `cd` into each folder and run `npm install` separately.

Stage 2 — Setting Up a Workspace

Now we convert this to a workspace. The entire setup requires only one change: add a `package.json` at the root with a `"workspaces"` field that tells npm where the sub-packages are.

Step 1 — Create the root `package.json`

my-app/package.json (root)

```
{
  "name": "my-app",
  "version": "1.0.0",
  "private": true,
  "workspaces": [
    "packages/*"
  ]
}
```

Two important things to note:

- `"private": true` prevents you from accidentally publishing the root package itself to the npm registry. This is always required for workspace roots.
- `"packages/*"` is a glob pattern — it tells npm that every folder inside `packages/` is a workspace package. You could also list them explicitly: `["packages/frontend", "packages/backend", "packages/shared"]`.

Step 2 — Create the three `package.json` files for each workspace

my-app/packages/shared/package.json

```
{  
  "name": "@my-app/shared",  
  "version": "1.0.0",  
  "main": "index.js"  
}
```

my-app/packages/backend/package.json

```
{  
  "name": "@my-app/backend",  
  "version": "1.0.0",  
  "main": "index.js",  
  "dependencies": {  
    "express": "^4.18.2",  
    "@my-app/shared": "1.0.0"  
  }  
}
```

my-app/packages/frontend/package.json

```
{  
  "name": "@my-app/frontend",  
  "version": "1.0.0",  
  "main": "index.js",  
  "dependencies": {  
    "@my-app/shared": "1.0.0"  
  }  
}
```

`@my-app/shared` is declared as a dependency of both `backend` and `frontend`. Notice that the version (`1.0.0`) matches exactly the version declared in the shared package's own `package.json`. npm workspaces will recognise this and create a **symlink** instead of downloading it from the registry — because it already exists locally inside `packages/shared/`.

Step 3 — Run one install from the root

```
cd my-app  
npm install
```

Output:

added 57 packages, and audited 60 packages in 2s
found 0 vulnerabilities

Now look at the structure. There is only one `node_modules` — at the root:

```
my-app/
├─ node_modules/                ← ONE shared node_modules for all packages
│   ├─ express/                 ← installed once, used by backend
│   └─ @my-app/
│       └─ shared → ../../packages/shared ← SYMLINK, not a copy
├─ package.json                 ← root (workspace config)
├─ package-lock.json            ← one lock file for the whole repo
└─ packages/
    ├─ shared/
    │   ├─ package.json
    │   └─ index.js
    ├─ backend/
    │   ├─ package.json
    │   └─ index.js
    └─ frontend/
        ├─ package.json
        └─ index.js
```

`express` is installed once. The `@my-app/shared` entry in `node_modules` is a *symlink* pointing directly to `packages/shared/`. This means when you change a file in `packages/shared/`, the change is immediately visible to backend and frontend — no reinstall, no republish needed.

Stage 3 — Writing Code Across Packages and Running Workspace Scripts

Now let us wire up actual code across packages to see the cross-package reference in action.

```
my-app/packages/shared/index.js
```

```
// Shared utilities used by both backend and frontend.
// Because it is a symlinked workspace package, both can require it
// using the package name – just like any npm package.

function formatName(firstName, lastName) {
  return `${firstName.trim()} ${lastName.trim()}`;
}

function getCurrentTimestamp() {
  return new Date().toISOString();
}

module.exports = { formatName, getCurrentTimestamp };
```

my-app/packages/backend/index.js

```
// The backend requires the shared package by its PACKAGE NAME,
// not by a relative path. npm workspaces makes this work via symlinks.
const { formatName, getCurrentTimestamp } = require('@my-app/shared');

const express = require('express');
const app = express();

app.get('/user', (req, res) => {
  const name = formatName('Alice', 'Smith');
  const timestamp = getCurrentTimestamp();

  res.json({ name, timestamp });
});

app.listen(3000, () => {
  console.log('Backend running on port 3000');
  console.log('Using shared formatName:', formatName('Node', 'JS'));
});
```

my-app/packages/frontend/index.js

```
// The frontend also uses the SAME shared package.
// If you update shared/index.js, BOTH packages benefit immediately.
const { formatName } = require('@my-app/shared');

const displayName = formatName(' Bob ', ' Jones ');
```

```
console.log('Frontend display name:', displayName);  
// Output: Frontend display name: Bob Jones  
// Note: shared's formatName trims whitespace – the fix applies everywhere.
```

To run scripts across all workspaces at once, use the `--workspaces` flag from the root:

```
# Run the "test" script in every workspace that has one  
npm run test --workspaces  
  
# Run a script in a SPECIFIC workspace (by package name)  
npm run start --workspace=packages/backend  
  
# Short form  
npm run start -w packages/backend  
  
# Install a package into a specific workspace (not the root)  
npm install lodash --workspace=packages/backend
```

Output:

```
> my-app/packages/backend npm run start
```

```
Backend running on port 3000  
Using shared formatName: Node JS
```

The key insight: Because `@my-app/shared` is a symlink to the actual source folder, there is no build step, no version bump, and no reinstall needed when you edit the shared code. The change propagates instantly. This is the main productivity advantage of workspaces over separate repositories.

3. Alternative Package Managers: Yarn and pnpm

npm is the default package manager that ships with Node.js and is the tool you have been using throughout this section. However, two well-established alternatives exist: **Yarn** and **pnpm**. You will encounter both frequently in professional codebases and open-source projects.

Neither is a replacement you must learn immediately. Understanding *why* they exist and *what they look like* is sufficient at this stage — you should not be confused or blocked when a project uses one of them.

3.1 Why Alternatives Exist

Both Yarn and pnpm were created because npm had significant problems at specific points in its history:

- **Yarn** was released by Facebook in 2016 when npm had no lock file (before `package-lock.json`), was slow, and gave inconsistent installs across machines. Yarn introduced lock files, parallel downloads, and a local cache — features npm later adopted.
- **pnpm** was created to solve a different problem: disk space and install speed. npm and Yarn both copy packages into each project's `node_modules`. If you have 10 projects all using the same version of `express`, it is stored on disk 10 times. pnpm uses a global content-addressable store and hard links — storing each package version once, globally, and linking to it from each project.

Today, npm has improved significantly and is production-grade for most use cases. The choice between them often comes down to team preference, existing project setup, or specific performance needs at scale.

3.2 Yarn

Yarn currently has two major versions in use: **Yarn Classic (v1)** and **Yarn Berry (v2+)**. Yarn Classic is the most widely encountered in existing codebases. Yarn Berry introduced a controversial feature called Plug'n'Play (PnP) that eliminates `node_modules` entirely — but it requires additional tooling support and is not universally adopted.

Installing Yarn:

```
npm install -g yarn    ← installs Yarn Classic (v1)

# Modern way – using corepack (bundled with Node.js 16.10+)
corepack enable
corepack prepare yarn@stable --activate
```

Core Yarn commands — compared to npm equivalents:

Action	npm	yarn
Initialise project	<code>npm init -y</code>	<code>yarn init -y</code>
Install all dependencies	<code>npm install</code>	<code>yarn</code> or <code>yarn install</code>
Add a package	<code>npm install express</code>	<code>yarn add express</code>
Add a dev dependency	<code>npm install jest -D</code>	<code>yarn add jest --dev</code>
Remove a package	<code>npm uninstall express</code>	<code>yarn remove express</code>
Run a script	<code>npm run start</code>	<code>yarn start</code>
Run a custom script	<code>npm run build</code>	<code>yarn build</code>
Upgrade packages	<code>npm update</code>	<code>yarn upgrade</code>
Lock file name	<code>package-lock.json</code>	<code>yarn.lock</code>

The most visible difference when working with a Yarn project: the lock file is called `yarn.lock` instead of `package-lock.json`. Both serve the same purpose — locking exact installed versions. When you clone a project that has a `yarn.lock` file, it signals that the project uses Yarn, and you should run `yarn` (not `npm install`) to maintain consistency.

Do not mix package managers in the same project. If a project uses Yarn and has a `yarn.lock` file, do not run `npm install` — it will create a `package-lock.json`, and now you have two conflicting lock files. Always use the same package manager the project was initialised with. You can identify which one a project uses by checking which lock file is present.

3.3 pnpm

pnpm (Performant npm) takes a fundamentally different approach to how packages are stored. Instead of copying files into each project's `node_modules`, it maintains a single global store on your computer and uses **hard links** to reference packages from that store.

Analogy

npm and Yarn are like a photocopier — every project gets its own physical copy of each book (package). If you have 10 projects using `lodash`, you have 10 physical copies. pnpm is like a library — there is one copy of each book, and every project gets a reading pass (hard link) to the same single copy. No duplication, much less disk space.

Installing pnpm:

```
npm install -g pnpm

# Or using corepack (recommended)
corepack enable
corepack prepare pnpm@latest --activate
```

Core pnpm commands — compared to npm and yarn:

Action	npm	yarn	pnpm
Install all dependencies	<code>npm install</code>	<code>yarn</code>	<code>pnpm install</code>
Add a package	<code>npm install express</code>	<code>yarn add express</code>	<code>pnpm add express</code>
Add a dev dependency	<code>npm install jest -D</code>	<code>yarn add jest --dev</code>	<code>pnpm add jest -D</code>
Remove a package	<code>npm uninstall express</code>	<code>yarn remove express</code>	<code>pnpm remove express</code>
Run a script	<code>npm run start</code>	<code>yarn start</code>	<code>pnpm run start</code> or <code>pnpm start</code>
Lock file name	<code>package-lock.json</code>	<code>yarn.lock</code>	<code>pnpm-lock.yaml</code>
Global store	No	Yes (cache)	Yes (hard-linked store)

pnpm also has excellent built-in workspace support, and is increasingly the preferred choice for monorepos — particularly in the Vue.js ecosystem and projects that need fast CI/CD pipelines.

4. Choosing the Right Tool

For the purposes of this course, all exercises use **npm**. However, when you join a team or open-source project, you should be able to identify which tool is in use and operate accordingly. The table below gives a practical summary:

Factor	npm	yarn	pnpm
Installation required	No — ships with Node.js	Yes — install separately	Yes — install separately
Lock file	<code>package-lock.json</code>	<code>yarn.lock</code>	<code>pnpm-lock.yaml</code>
Disk space efficiency	Low — copies per project	Medium — local cache	High — global hard-linked store
Install speed	Good	Good (parallel)	Fastest (hard links, no copying)
Monorepo (workspaces)	Yes (npm v7+)	Yes (Yarn Classic & Berry)	Yes — highly optimised
Industry adoption	Universal default	Common in React/ Meta ecosystem	Growing fast — Vue, Vite, modern tooling
Best for	Learning, most projects, teams without strong preference	Projects already using Yarn, Meta/React toolchain	Monorepos, CI speed, disk-constrained environments

How to identify which package manager a project uses:

- Has `package-lock.json` → use **npm**
- Has `yarn.lock` → use **yarn**
- Has `pnpm-lock.yaml` → use **pnpm**
- Has a `"packageManager"` field in `package.json` → use whichever it specifies

The "packageManager" field

Modern `package.json` supports a "packageManager" field that explicitly declares which tool and version should be used with the project. Combined with `corepack` (bundled with Node.js), this enforces the correct tool automatically:

`package.json`

```
{
  "name": "my-app",
  "version": "1.0.0",
  "packageManager": "pnpm@9.1.0"
}
```

When `corepack` is enabled and a developer runs `npm install` in this project, `corepack` intercepts and warns them to use `pnpm` instead. This prevents accidental mixing of package managers on a team.

Summary

Concept	Key Point
Monorepo	Multiple related packages in a single Git repository. Enables shared code without a publish cycle and atomic cross-package changes.
npm Workspaces	Built-in npm feature (v7+) that manages monorepos. Declared via the "workspaces" field in the root <code>package.json</code> . Hoists shared deps to root <code>node_modules</code> and creates symlinks for local packages.
<code>"private": true</code>	Required on the root <code>package.json</code> of a workspace. Prevents accidental publishing of the root to the npm registry.

Concept	Key Point
Local package reference	When a workspace package is listed as a dependency of another workspace package (matching name and version), npm creates a symlink instead of downloading from the registry.
Yarn	Alternative package manager. Very similar commands to npm. Uses <code>yarn.lock</code> . Common in React/Meta ecosystem. Introduced lock files before npm had them.
pnpm	Fastest package manager. Uses a global hard-linked store — packages are stored once per version on disk. Uses <code>pnpm-lock.yaml</code> . Growing rapidly in modern tooling.
Identifying the tool	Check the lock file in the project: <code>package-lock.json</code> → npm, <code>yarn.lock</code> → yarn, <code>pnpm-lock.yaml</code> → pnpm. Never mix tools in the same project.

Section 2 Complete — Full Recap

This section covered the three subtopics of NPM and Package Management. Below is a consolidated reference of every command introduced across all three subtopics:

Command	What it does
<code>npm init -y</code>	Initialise a new project with default package.json
<code>npm install <pkg></code>	Install a package locally as a production dependency

Command	What it does
<code>npm install <pkg> -D</code>	Install as a dev dependency
<code>npm install -g <pkg></code>	Install a CLI tool globally
<code>npm install</code>	Restore all packages from package.json + lock file
<code>npm uninstall <pkg></code>	Remove a package
<code>npm run <script></code>	Run a custom script from package.json
<code>npm outdated</code>	Check for available package updates
<code>npm update</code>	Update packages within their declared version ranges
<code>npm audit</code>	Check for known security vulnerabilities
<code>npm audit fix</code>	Auto-fix vulnerabilities within safe version ranges
<code>npm install --omit=dev</code>	Install only production dependencies (for deployment)
<code>npm run <script> --workspaces</code>	Run a script in all workspace packages
<code>npm run <script> -w <pkg></code>	Run a script in a specific workspace package

Interview Questions

1. What is a monorepo and what problem does it solve compared to separate repositories?

Expected answer: A monorepo stores multiple related packages (e.g., frontend, backend, shared library) in a single Git repository. It solves the problem of keeping related code in sync

— shared code changes are immediately available to all packages without a publish/update cycle. It also allows atomic commits that span multiple packages and simplifies CI/CD pipelines.

2. What does npm Workspaces do with `node_modules` when you run `npm install` at the root?

Expected answer: npm hoists shared dependencies to a single `node_modules` folder at the root of the repository, avoiding duplication. For local workspace packages listed as dependencies of each other, npm creates symlinks in `node_modules` pointing to the actual source folders — so code changes propagate immediately without reinstallation.

3. Why is `"private": true` required in the root `package.json` of a workspace?

Expected answer: It prevents the root package from being accidentally published to the npm registry. The root is a coordination layer — it has no code of its own to publish. Without this flag, running `npm publish` from the root could publish an empty or misconfigured package.

4. You clone a project and see a `yarn.lock` file. What should you do, and what should you *not* do?

Expected answer: You should run `yarn` or `yarn install` to install dependencies. You should not run `npm install` — doing so would create a `package-lock.json`, introducing a second lock file. Two lock files in the same project create version inconsistencies and confusion for the rest of the team.

5. What is the fundamental difference between how pnpm stores packages compared to npm?

Expected answer: npm copies the package files into each project's `node_modules` — if 10 projects use the same version of `express`, there are 10 copies on disk. pnpm maintains a single global content-addressable store and uses hard links (file system references) from each project's `node_modules` to that single copy. Each package version is stored only once globally, regardless of how many projects use it — saving significant disk space and making installs faster.

6. In a npm workspace, how does a package reference another local package without publishing it to the registry?

Expected answer: By listing the local package as a dependency in `package.json` with the matching package name and version. When `npm install` runs at the workspace root, npm

detects that the dependency's name and version match an existing workspace package and creates a symlink in `node_modules` pointing to that local folder, instead of downloading from the registry.

Practical Assignment

Title: Build a Minimal Monorepo with npm Workspaces

Objective: Set up a working workspace with a shared utility package consumed by two other packages.

Instructions:

1. Create the following folder structure manually:

```
monorepo-demo/  
├─ package.json      ← root workspace config  
└─ packages/  
    ├─ utils/         ← shared utilities  
    ├─ app-a/         ← uses utils  
    └─ app-b/         ← also uses utils
```

2. Set up the root `package.json` with `"private": true` and a `"workspaces"` field pointing to `packages/*`.
3. Create a `package.json` for each workspace:
 - `utils` → name: `@demo/utils`, version: `1.0.0`
 - `app-a` → name: `@demo/app-a`, depends on `@demo/utils` and `chalk`
 - `app-b` → name: `@demo/app-b`, depends on `@demo/utils` and `dayjs`
4. Write `utils/index.js` that exports two functions: `greet(name)` which returns `"Hello, <name>!"`, and `shout(text)` which returns the text in ALL CAPS.
5. Write `app-a/index.js` that uses both `@demo/utils` and `chalk` to print a coloured greeting.
6. Write `app-b/index.js` that uses `@demo/utils` and `dayjs` to print the shouted text and today's date.

7. Add a `"start"` script to each app's `package.json` that runs its `index.js`.

8. Run `npm install` from the root, then confirm:

- There is only one `node_modules` at the root (not inside each package)
- `node_modules/@demo/Utils` is a symlink
- `npm run start -w packages/app-a` works correctly
- `npm run start -w packages/app-b` works correctly

9. Modify the `greet()` function in `utils/index.js` to add an exclamation mark at the end. Re-run both apps and confirm the change reflected in *both* without any reinstall.

Bonus challenge: Add a `"start:all"` script to the root `package.json` that runs the start script in all workspaces using `--workspaces`.